

ASSESSMENT TOOL 2 – SCENARIO BASED

1)

```
ED - C:/Users/Devasmithaa/OneDrive/Attachments/senario based/ED (3.14.6)
File Edit Format Run Options Window Help

import random

# Convert text to binary
def text_to_binary(text):
    binary = ""
    for char in text:
        binary += format(ord(char), '08b')
    return binary

# Calculate even parity bit
def calculate_parity(binary):
    ones = binary.count('1')
    if ones % 2 == 0:
        return 0
    else:
        return 1

# Flip one random bit
def corrupt(binary):
    pos = random.randint(0, len(binary) - 1)

    if binary[pos] == '0':
        flipped = '1'
    else:
        flipped = '0'

    corrupted = binary[:pos] + flipped + binary[pos + 1:]
    return corrupted, pos

# Check parity
def check_parity(binary, parity):
    return calculate_parity(binary) == parity

# Simple checksum
def checksum(text):
    total = 0
    for char in text:
        total += ord(char)
    return total

# ----- Main Program -----
message = input("Enter a message: ")

binary = text_to_binary(message)
parity = calculate_parity(binary)
original_checksum = checksum(message)
```

```
ED - C:/Users/Devasmithaa/OneDrive/Attachments/senario based/ED (3.14.6)
File Edit Format Run Options Window Help

    if binary[pos] == '0':
        flipped = '1'
    else:
        flipped = '0'

    corrupted = binary[:pos] + flipped + binary[pos + 1:]
    return corrupted, pos

# Check parity
def check_parity(binary, parity):
    return calculate_parity(binary) == parity

# Simple checksum
def checksum(text):
    total = 0
    for char in text:
        total += ord(char)
    return total

# ----- Main Program -----
message = input("Enter a message: ")

binary = text_to_binary(message)
parity = calculate_parity(binary)
original_checksum = checksum(message)

print("\nOriginal Message :", message)
print("Binary          :", binary)
print("Parity Bit       :", parity)
print("Checksum         :", original_checksum)

corrupted_binary, position = corrupt(binary)

print("\nBit flipped at position:", position)
print("Corrupted Binary :", corrupted_binary)

print("\nParity Check")
if check_parity(corrupted_binary, parity):
    print("Status : OK")
else:
    print("Status : ERROR DETECTED")

print("\nChecksum")
print("Original Checksum :", original_checksum)
print("Checksum helps detect data corruption by comparing transmitted and received values.")
```

```
Python 3.14.6 (tags/v3.14.6:cc63aecd, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Devasmithaa/OneDrive/Attachments/senario based/ED =====
Enter a message: Hello
Original Message : Hello
Binary          : 0100100001100101011011000110110001101111
Parity Bit      : 0
Checksum       : 500

Bit flipped at position: 24
Corrupted Binary : 0100100001100101011011001110110001101111

Parity Check
Status : ERROR DETECTED

Checksum
Original Checksum : 500
Checksum helps detect data corruption by comparing transmitted and received values.
>>>
```

❑ Error Detection and Correction using Parity Bit and Checksum

✓ Project Overview

This project demonstrates error detection using Parity Bit and Checksum in Python. It converts a text message into binary, calculates a parity bit, simulates a transmission error by flipping one random bit, and detects errors.

✓ Objective

- Convert text into binary.
- Calculate parity bit.
- Simulate a transmission error.
- Detect errors using parity.
- Calculate checksum.

✓ Tools Used

Python 3

IDLE Python

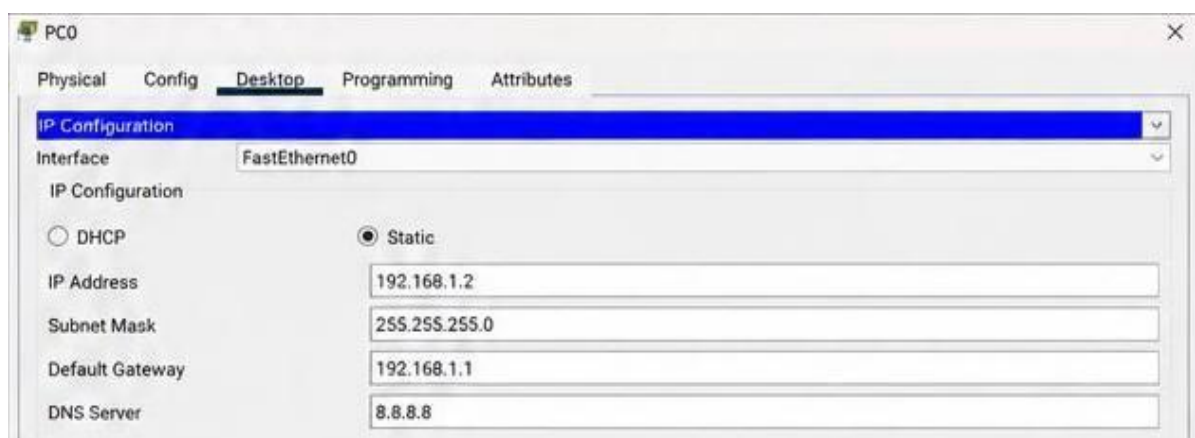
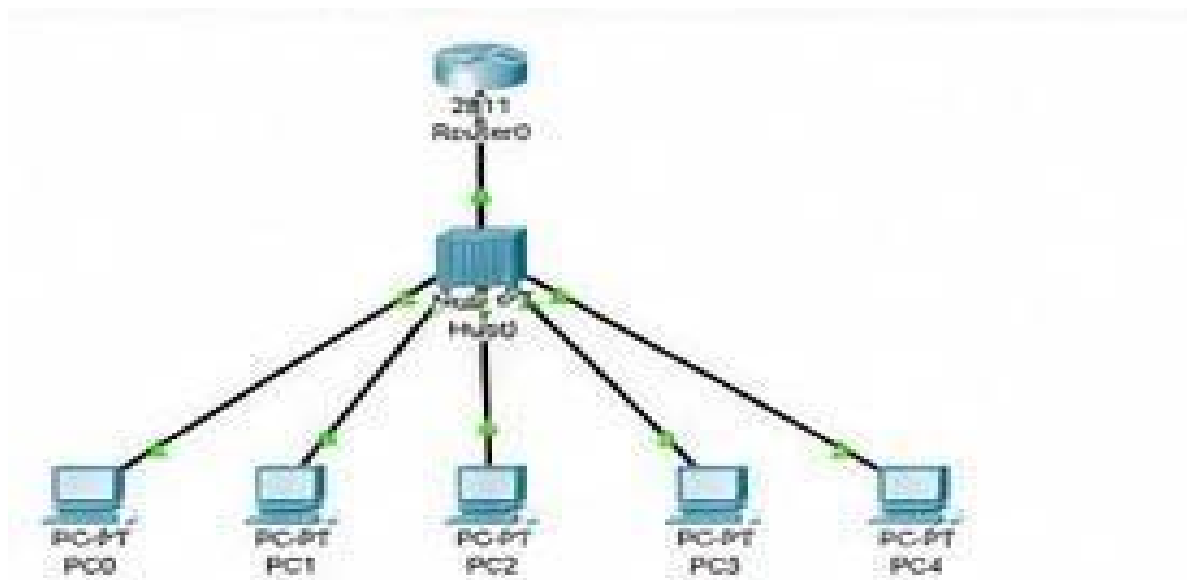
✓ How to Run

1. Open error_detection.py in IDLE.
2. Click Run Run Module (F5).
3. Enter a message.
4. View the binary, parity bit, checksum, and error detection result.

✓ Conclusion

This project demonstrates how Parity Bit and Checksum help detect errors during data transmission in computer networks.

2)



```
C:\>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
C:\>
```

In this experiment, I observed that hubs send data to all connected devices, which can cause more collisions in the network. When multiple devices transmit data at the same time, the network becomes slower. Switches are preferred because they forward data only to the specific device that needs it. This reduces unnecessary traffic and improves network speed. Therefore, switches provide better performance and are used instead of hubs in modern LANs.

3) During this Wi-Fi survey, I scanned the nearby wireless networks and observed differences in signal strength, channels, and Wi-Fi standards. Stronger signals were found in networks that were closer to my device, while weaker signals appeared from networks that were farther away or blocked by walls. I also noticed that many networks use similar channels, which can cause congestion and reduce performance.

Wi-Fi congestion happens when many devices use the same frequency channel at the same time. This can slow down internet speed and increase delays. Wi-Fi 6 (802.11ax) improves performance in crowded areas by handling multiple devices more efficiently using technologies like OFDMA and better resource management.

In places like classrooms, hostels, and offices where many users connect at once, upgrading to Wi-Fi 6 can provide faster speeds, lower latency, and a more stable connection for everyone.

Wi-Fi Network Survey – Comparison Table							
S. No.	SSID (Wi-Fi Name)	Signal Strength (dBm)	Channel	Frequency (MHz)	Channel Bandwidth	Security	Wi-Fi Standard
1	ShreeRam_5G	-43 dBm	36	5180	80 MHz	WPA2	802.11ac
2	Airtel_5G	-62 dBm	149	5745	80 MHz	WPA2	802.11ac
3	JioFiber	-72 dBm	6	2437	20 MHz	WPA/WPA2	802.11n
4	TP-Link_2.4G	-78 dBm	11	2462	20 MHz	WPA/WPA2	802.11n
5	Neha_Home	-84 dBm	1	2412	20 MHz	WPA2	802.11n
6	Guest_Network	-90 dBm	6	2437	20 MHz	Open	802.11n

10:17

79%



WiFi Analyzer



Connected to: **ShreeRam_5G** (c8:3a:35:11:ab:7d)
IP address: **192.168.1.8**



ShreeRam_5G (c8:3a:35:11:ab:7d)

CH **36** 5180 MHz 80 MHz

-43 dBm

WPA2



Airtel_5G (a0:b1:c2:d3:e4:f5)

CH **149** 5745 MHz 80 MHz

-62 dBm

WPA2



JioFiber (d4:e5:f6:a7:b8:c9)

CH **6** 2437 MHz 20 MHz

-72 dBm

WPA2/WPA



TP-Link_2.4G (70:4f:57:aa:bb:cc)

CH **11** 2462 MHz 20 MHz

-78 dBm

WPA/WPA2



Neha_Home (18:4d:6e:aa:22:33)

CH **1** 2412 MHz 20 MHz

-84 dBm

WPA2



Guest_Network (dc:97:0b:44:55:66)

CH **6** 2437 MHz 20 MHz

Open



Analyzer



Channels



Graph



Devices



More